

Software Engineering – 5CM505

Lecture 4

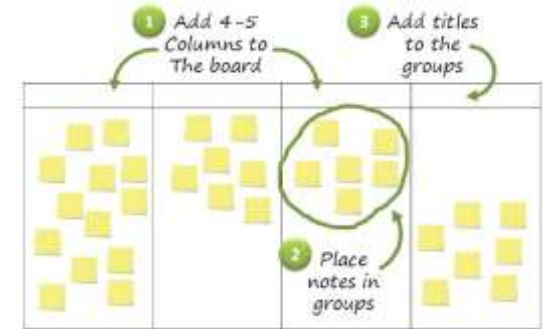
Modelling Tools and Techniques

Wajahat Ali Khan



Content

- Recap from last week
- Requirements Processes and Tools
 - Systemic Textual Analysis
 - Viewpoint Analysis
 - Functional Modelling
 - Use cases
- Summary



Systemic Textual Analysis			
Project:		Date:	
Author:		Issue:	
Requirements		Comments	
Context:			
Operational Requirement:			
Non-functional System Requirements:			
Non-functional Implementation Requirement	Functional Requirement	Non-functional Performance Requirement	

Recap from Last Week

Requirements

- Definition
- Importance
- Characteristics of good requirements
- Stakeholder Identification
- Prioritization
 - MOSCOW method
- Categorization
 - Holistic Model
 - FURPS
 - FURPS+



Learning Outcomes

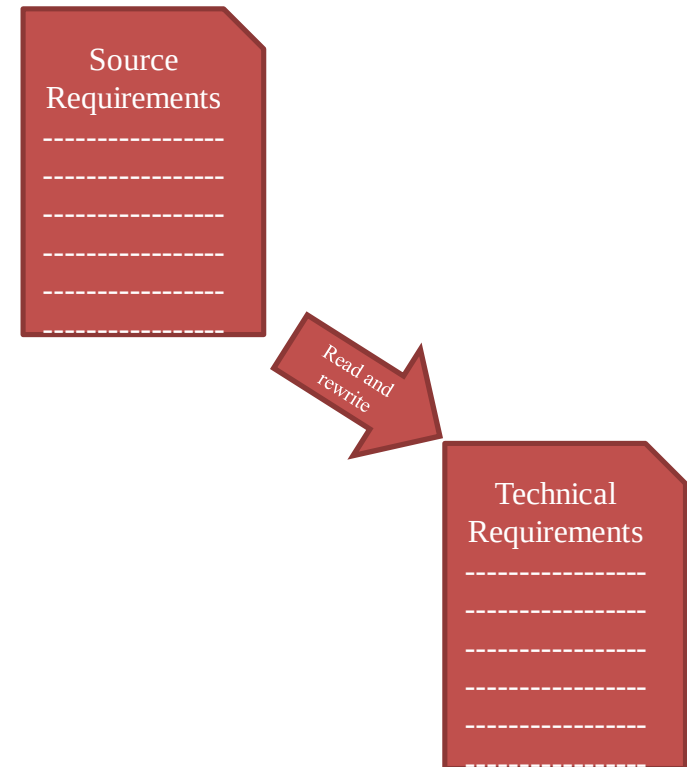
- By the end of this lecture you should be able to:
 - Use Systemic Textual Analysis to interpret, expand and clarify a set of source requirements
 - Use the above to identify missing requirements
 - Use Viewpoint Analysis to identify and structure a system's requirements
 - Explain the principles behind functional modelling, discuss its advantages and disadvantages, and use it to aid with designing software systems
 - Discuss the advantages and disadvantages of these two methods



Systemic Textual Analysis

Textual Analysis

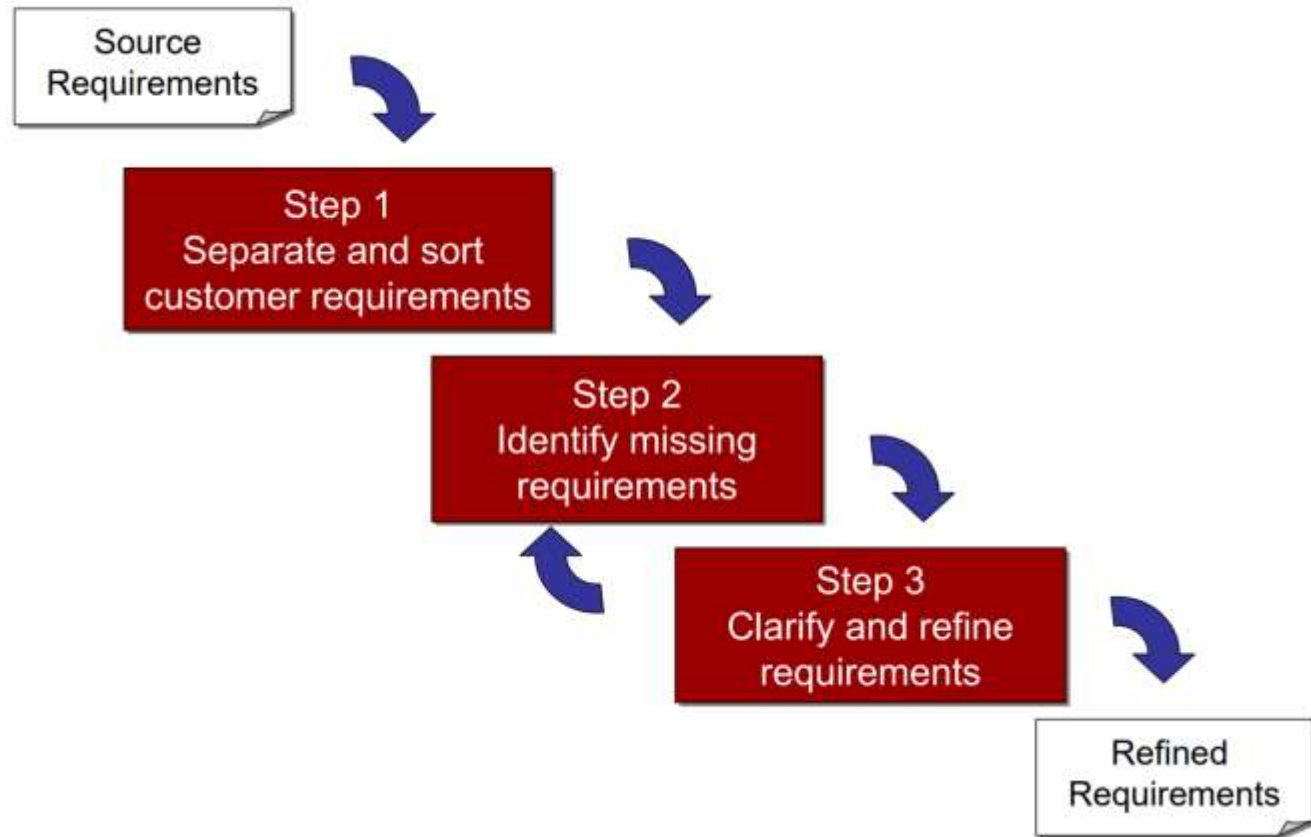
- Customer / stakeholder requirements (source requirements) are typically **hand-written documents** that are passed to software developers to interpret in some way so that a set of technical requirements can be produced
- The traditional method for understanding the source requirements is **Textual Analysis**
- This is a **line-by-line assessment** of the hand-written documents.
- In the assessment, the requirements are clarified and expanded where necessary
- However, there have been problems with this approach



Systemic Textual Analysis

- A **line-by-line assessment of the source requirements** but using the **Holistic Requirements Model** as a framework for organising the analysis
- Helps to interpret, expand and clarify the source requirements
- Identifies missing requirements
- The requirements are separated out into the following categories:
 - Operational
 - Functional
 - Non-functional
- Missing and ambiguous requirements are more easily identified

Systemic Textual Analysis



Clarifying and Refining Requirements

- Each requirement should be examined one at a time and the following questions should be asked
 - Is the requirement correct? (Is it asking for something possible?)
 - Is it complete? (Is it a full sentence?)
 - Is it clear? (Is it unambiguous and written in simple direct language?)
 - Is it consistent with the other requirements?
 - Is it verifiable? (Is there a way to demonstrate that it has been met?)
 - Is the prioritisation reasonable? (Customers often use “must” when “should” is more appropriate)



Systemic Textual Analysis Form

- Reliability
 - Maintainability
 - Interoperability
 - Efficiency
 - Usability
 - Security
 - Availability
 - Scalability
 - Stability

Systemic Textual Analysis			
Project:		Date:	
Author:		Issue:	
Requirements		Comments	
Context:			
Operational Requirement:			
Non-functional System Requirements:			
Non-functional Implementation Requirement	Functional Requirement	Non-functional Performance Requirement	

Identifying Missing Requirements

Non-functional implementation requirement	Functional requirement	Non-functional performance requirement
	Display summary sheet on click of the checkout button	Missing requirement
	Process PayPal transaction on click of the pay button	Missing requirement
	Process Visa transaction on click of the pay button	Missing requirement
List boxes shall be used	Missing requirement ?	
World Pay shall be used	Missing requirement ?	
	Missing requirement ?	Results returned from database within 5 seconds

The layout of the form helps to identify missing requirements

Group Exercise

- Imagine that you work for a company that sells books. Your Managing Director called a meeting last week announcing the development of new e-commerce software so that customers can make online purchases of your stock
- Come up with a set of requirements for that piece of software, bearing in mind the characteristics of good requirements
- Prioritise the set of requirements using the MoSCoW method.

- Categorise the e-commerce set of requirements you have been working on into:
 - An operational requirement
 - Functional requirements
 - Non-functional performance requirements
 - Non-functional system requirements
 - Non-functional implementation requirements

- Work in groups
- If you have your sheet from last week, see if you can organise your requirements into an STA form, and identify any missing ones
- OR - you can try to provide the missing requirements identified in the example

(10 minutes)



Deducing Requirements

- Systemic Textual Analysis can identify **where** requirements are missing
- We also need to deduce **what** the missing requirements are
- All gaps need to be **discussed with the customer(s) and stakeholders** to determine suitable technical requirements



Systemic Textual Analysis

ADVANTAGES

- Fairly simple and anyone can contribute to the process
- If performed in a team it can promote discussion and thinking
- It helps to structure the requirements
- It helps identify missing and ambiguous requirements

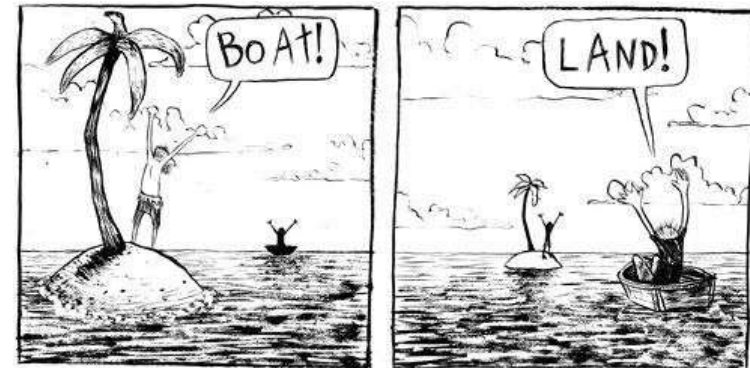
DISADVANTAGES

- Not useful if the requirements set is very small
- Can start with a list and end up with a bigger one
- Difficult to verify completeness and consistency
- Takes practice to recognise the different categories
- **No way to convey the relationships between requirements**

Viewpoint Analysis

Viewpoint Analysis

- A tool to help a **multi-disciplinary team** identify and structure the requirements of a system
- **Shows relationships** between individual requirements
- Particularly useful if there is only a **small number of requirements**
- The approach consists of brainstorming to generate and **structure** the requirements
- The system or application is examined from two viewpoints
 - **Functional Viewpoint**
 - **Non-functional Viewpoint**
- The structure should demonstrate how the **non-functional requirements impact upon the functional requirements**



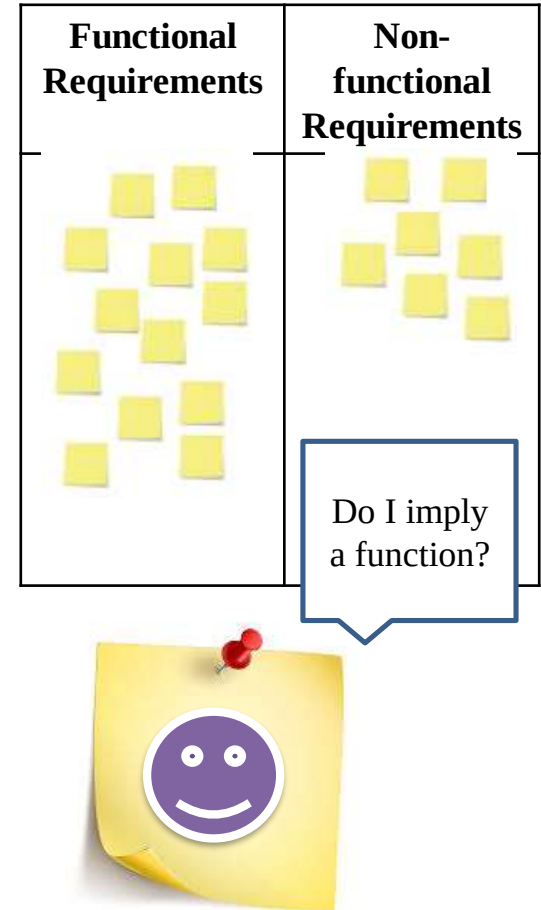
Step 1 : Requirements Generation

- The team “**brainstorm**”, writing down all possible requirements, for example on post-it notes
- Requirements could come from the experience of the team, an original customer requirements document or from legislative documents
- Note that it is **requirements that should be written down, not problem solutions**. In practice this is quite difficult, as people naturally tend to think in terms of solutions.
- There is thus a need to convert written solutions into logical requirements. This can be done as the brainstorm progresses or after the brainstorm has finished



Step 2 : Viewpoint Separation

- In this step the **functional and non-functional requirements are separated**
- Remember that customers are very focused upon specifying solutions (NFI) and performance (NFP and NFS). **Every post-it note should be checked to consider whether it implies a function**
- If a **function is deduced** then it should be recorded on the a new post-it note
- The aim is to identify all possible functionality for the proposed system
- **We can always choose not to implement it later, but if it is not identified we will not have the opportunity to make that decision**



Case Study



**A navigation system
for a mobile robot**

Example: Viewpoint Separation

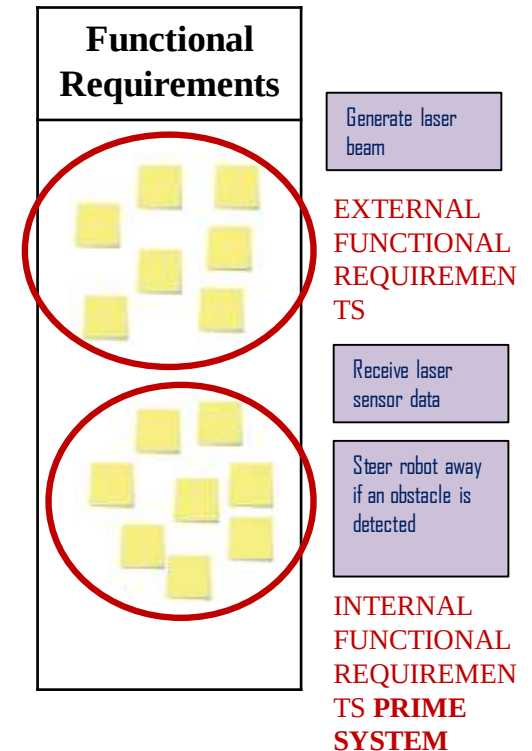
Functional Requirements	Non-functional Requirements
Detect obstacles	
Steer robot away if an object is detected	Use SICK laser to detect obstacles
Slow robot down if an object is detected	Detection of obstacles should be within 1° angular resolution
Generate laser beam	
Receive laser sensor data	



Functional requirement is implied.
Need to create a new functional post-it note

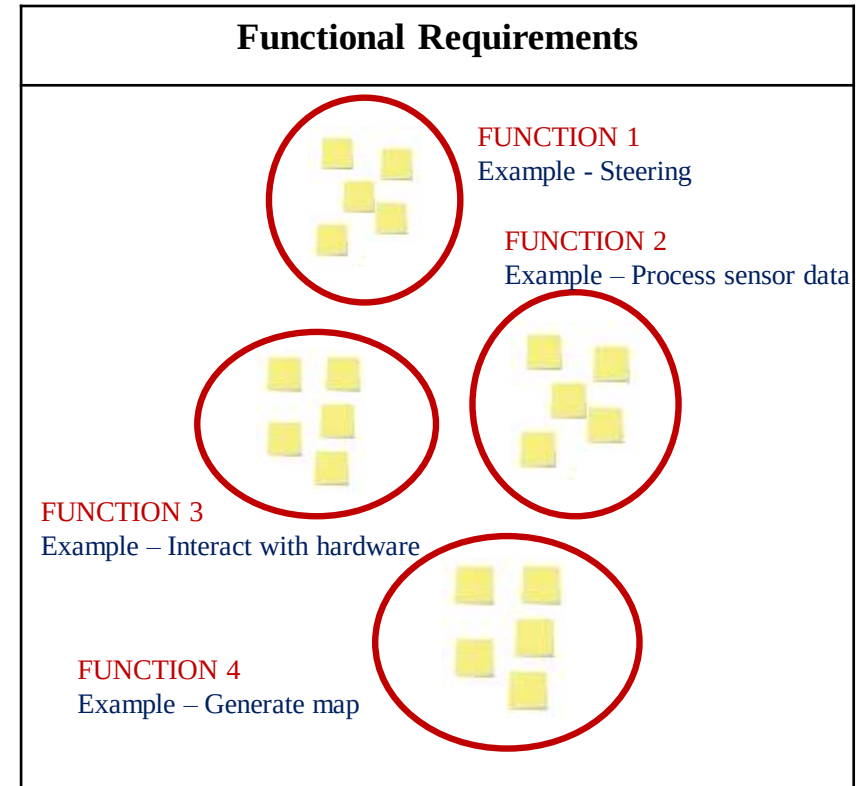
Step 3 : Functional Viewpoint Structuring

- This step **focuses on the functional requirements** so the non-functional requirements can be temporarily put aside
- The aim of the step is to structure the functional requirements logically
- First, the functional requirements are divided into two categories:
 - **External functional requirements**
 - **Internal functional requirements**
- This helps to **identify the prime system and the systems in the environment that interact with it**



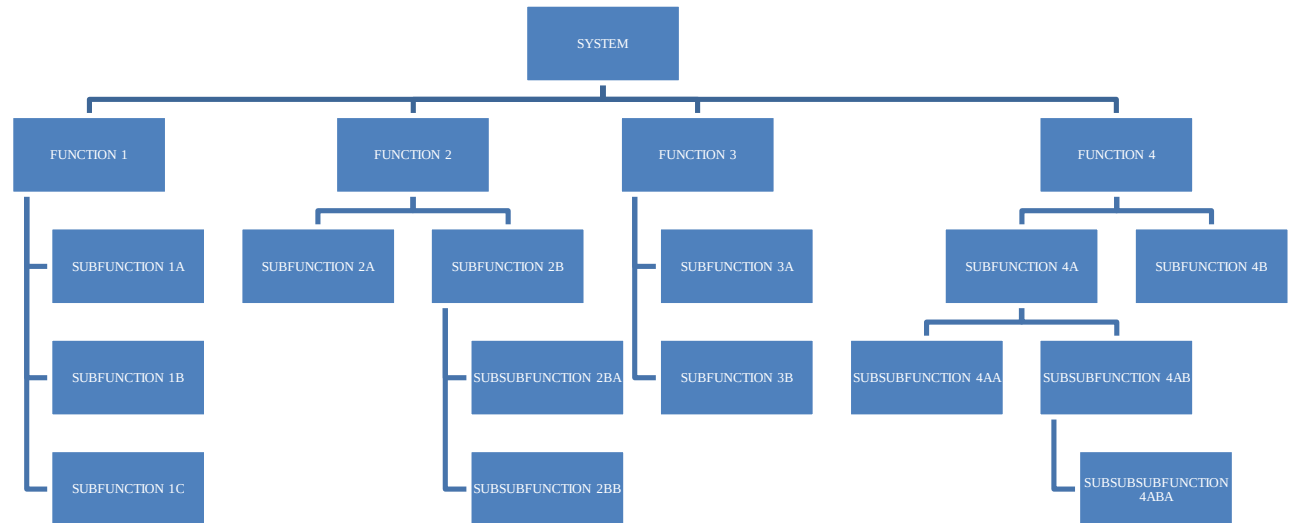
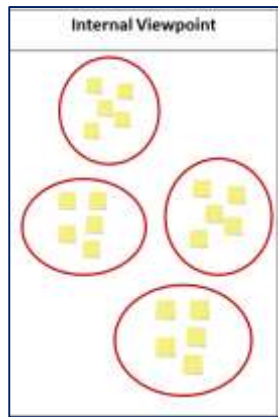
Step 4 : Functional Viewpoint Grouping

- In this step both the external and internal functions are grouped together logically , with a maximum of about 5 functions per group
- Each group is named. If there is difficulty naming it then it is likely not a logical grouping, so some reshuffling may be needed



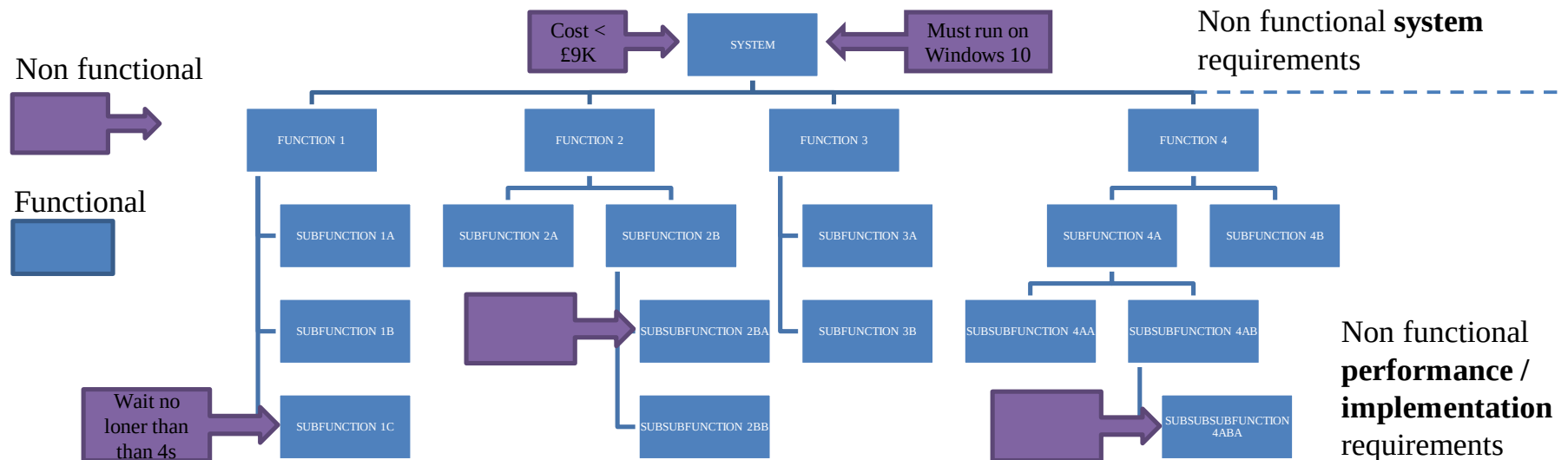
Step 5 : Viewpoint Structure Chart

- The structured notes are next converted into a hierarchically structured chart



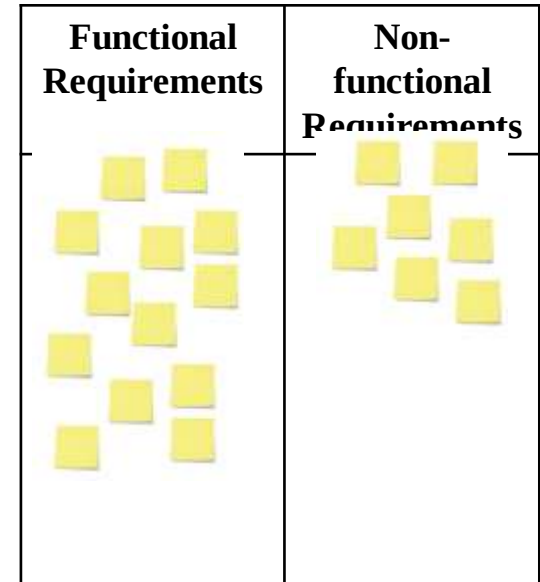
Step 6 : Non-functional Viewpoint Structure

- The final stage is to align the non-functional requirements with the functional requirements in the structural diagram



Viewpoint Analysis

- The purpose is to generate more knowledge about the proposed system, so it does not matter if the structure is different to another potential viewpoint structure
- The technique is generally very good at identifying 80% - 90% of system functionality
- Sometimes something else needs to be done to find the missing 10% - 20%, for example Systemic Textual Analysis or Functional Modelling



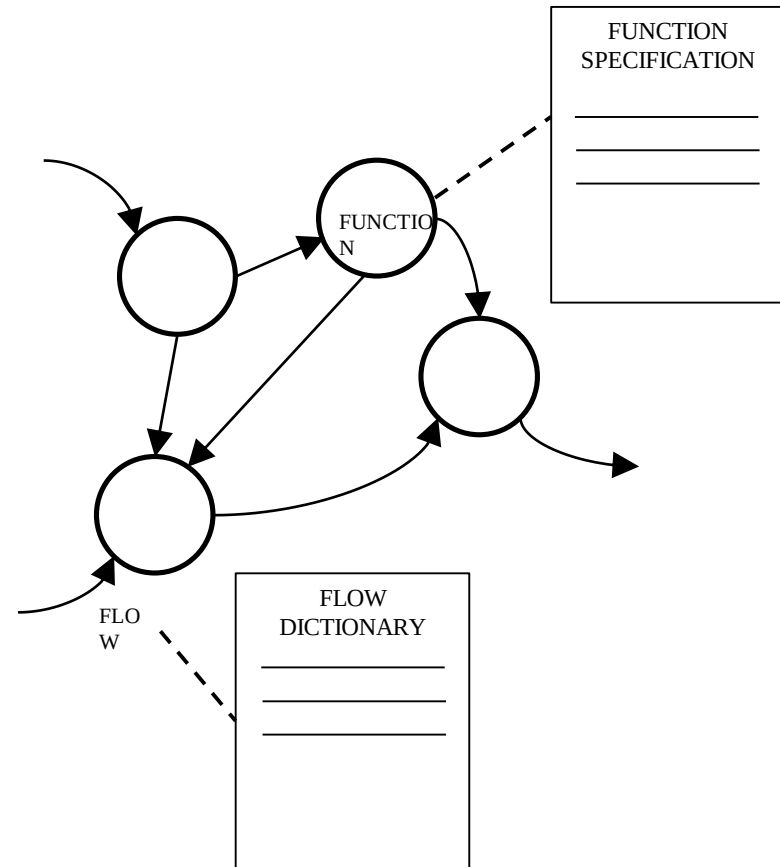
Summary of the Steps

- **Step 1. Brainstorming**
 - Write down all possible requirements, for example on post-it notes
- **Step 2. Viewpoint Separation**
 - The functional and non-functional requirements are separated and missing requirements are identified
- **Step 3. Functional Viewpoint Structuring**
 - The functional requirements are divided into external and internal functional requirements
- **Step 4. Functional Viewpoint Grouping**
 - Both the external and internal functions are grouped together logically and each group is named
- **Step 5. Viewpoint Structure Chart**
 - The groups are converted into a hierarchically structured chart
- **Step 6. Non-functional Viewpoint Structure**
 - The non-functional requirements are aligned with the functional requirements in the structural diagram

Functional Modelling

Functional Modelling

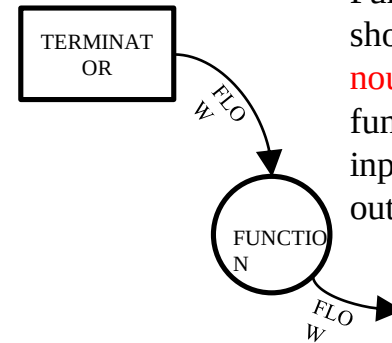
- A **tool** that highlights the functionality and the **inputs and outputs** to the identified functions
- Uses three sub-tools:
 - **Function Flow Diagram (FFD)**
 - **Flow Dictionary**
 - **Function Specification**



Function Flow Diagram (FFD)

- An abstract network representation of the system
- Depicts the system in terms of its component functions and the interdependencies or “flows” between them
- Limited diagramming conventions. Consists of only FUNCTIONS, TERMINATORS and FLOWS

Terminator name can be either a **verb-noun phrase** or a **noun**. A terminator represents the source or destination of flows **external** to the system



Function name should be a **verb-noun phrase**. The function transforms input flows into output flows

Flow name should be a **noun**. The flow represents an input or output

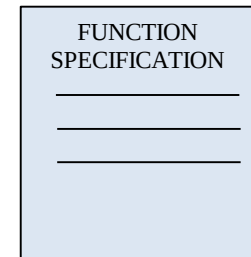
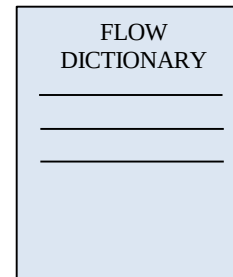
Flow Dictionary and Functional Specification

FLOW DICTIONARY (FD)

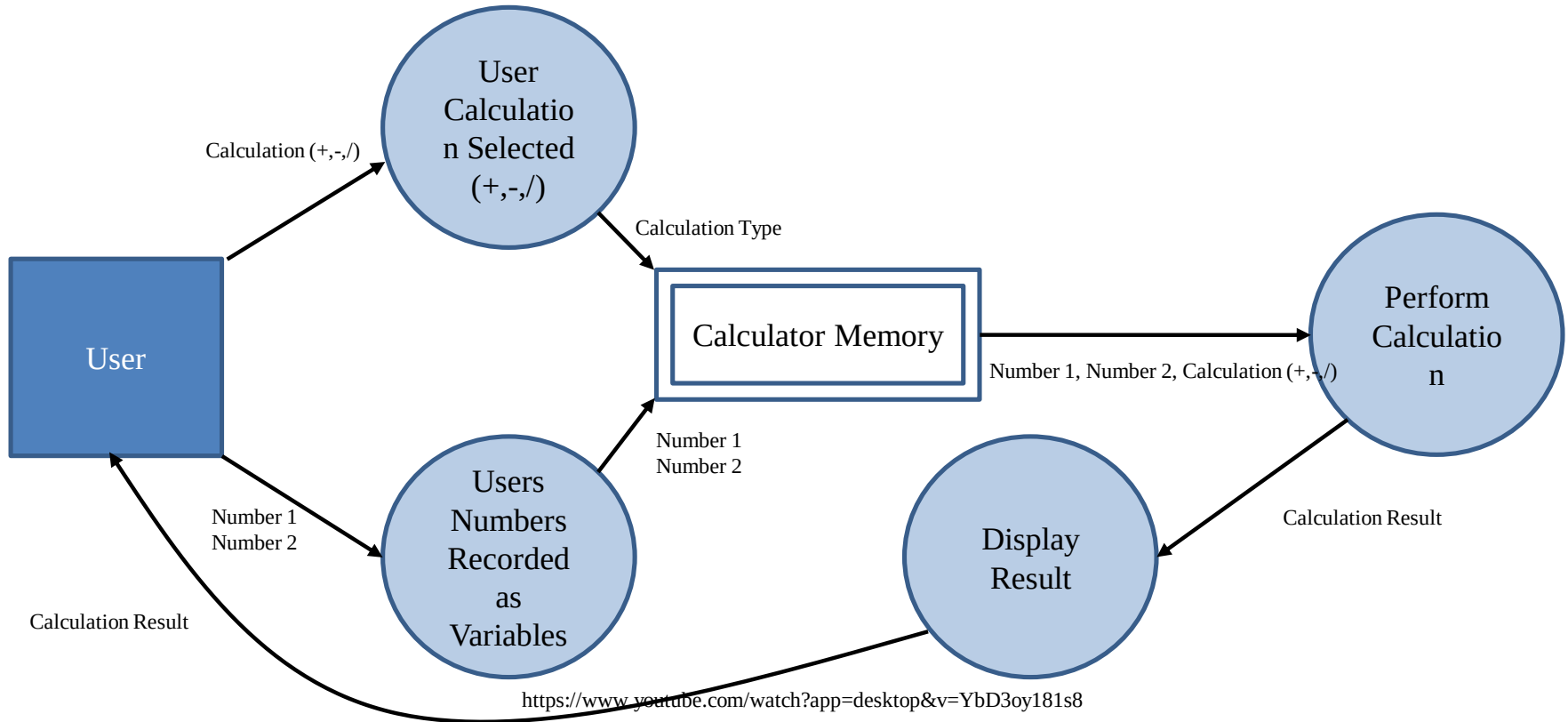
- Documents each of the flows on the diagram
- Declares the component elements of each input or output flow
- Also declares the relationships between them
- Specifies precisely what is meant by each flow on the FFD

FUNCTIONAL SPECIFICATION (FS)

- Specifies the functions by defining the transformation that converts inputs to outputs

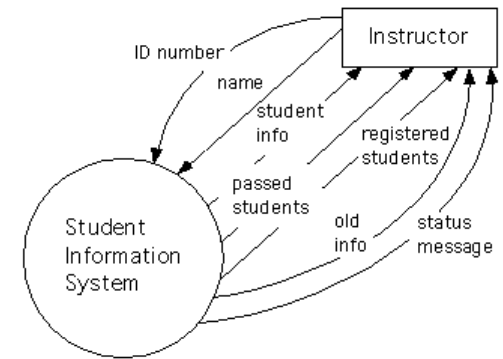


Functional Flow Diagram – Calculator Example



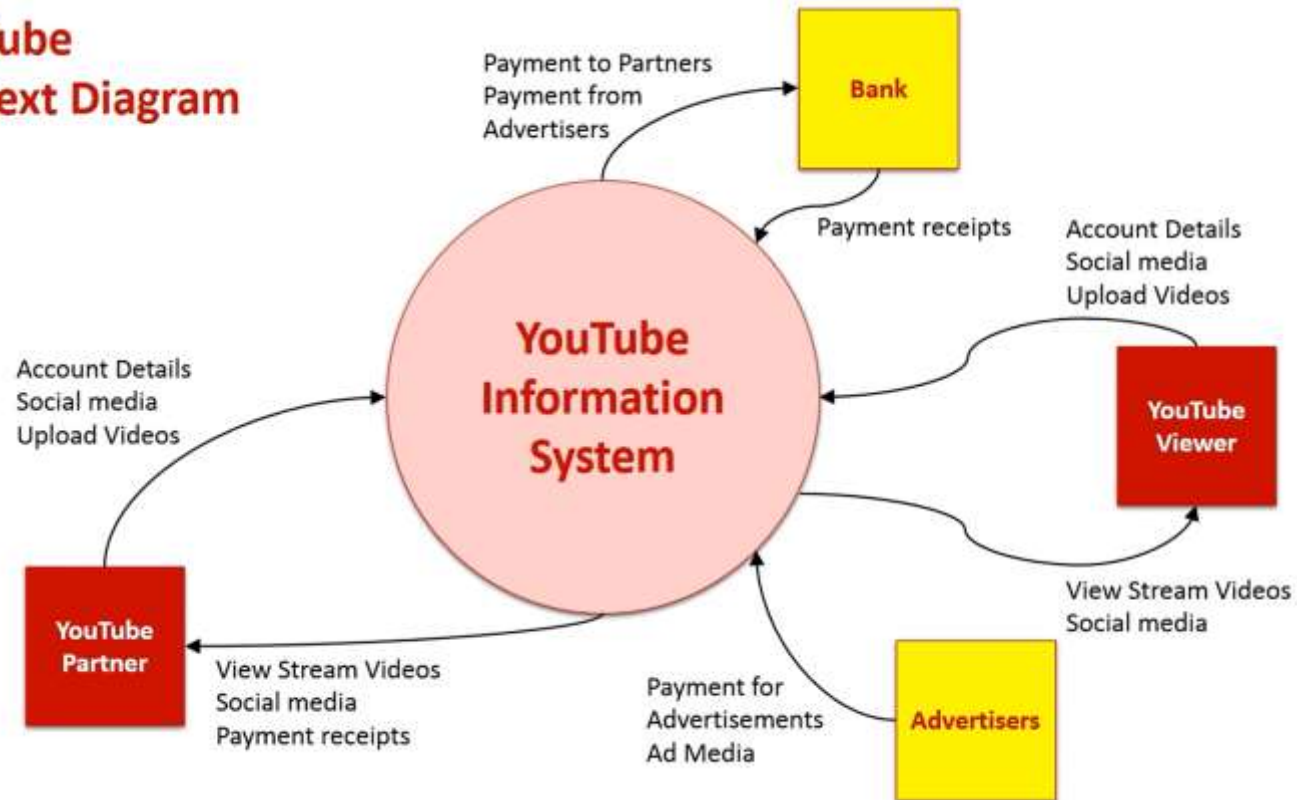
Context Diagram

- For large systems often a set of FFDs is required
- Can start with the **top level FFD**, which is referred to as the *Context Diagram*
- The Context Diagram shows the system in its **operational context – the “super-system”**. It has only one function
- It can be **decomposed into a number of functions** that can be represented together on one diagram. This is called **DIAGRAM 0**. Rule of thumb – no more than 7 functions in one diagram
- The functions in DIAGRAM 0 can be taken in turn and decomposed further into **lower-level FFDs**
- Can have a hierarchy of FFDs, each level providing lower level detail for the functions in the level directly above



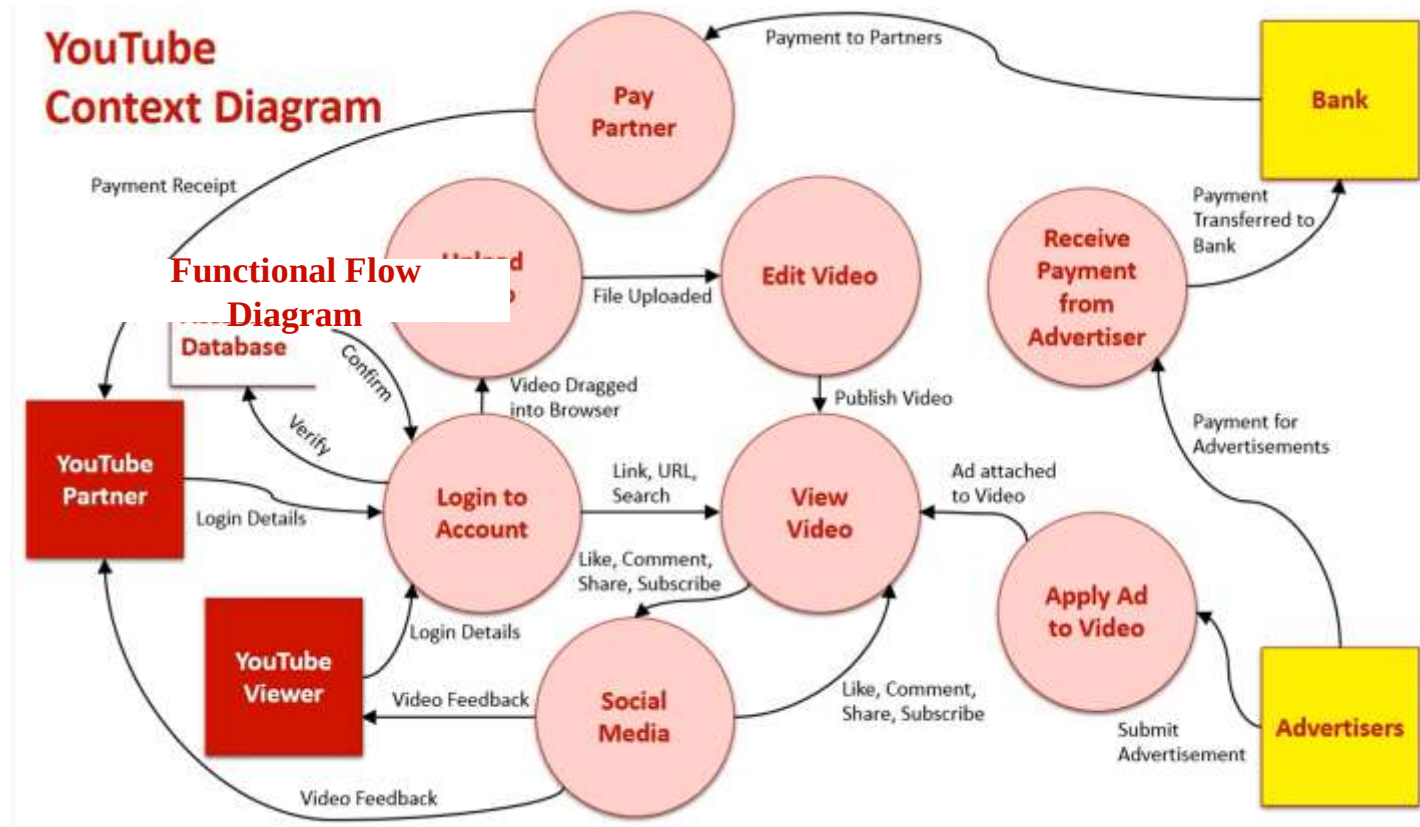
Context Diagram – YouTube Example

YouTube Context Diagram



<https://www.youtube.com/watch?v=hiMeEswjWuk>

Functional Flow Diagram – YouTube Example



<https://www.youtube.com/watch?v=hiMeEswjWuk>

Functional Modelling

ADVANTAGES

- Shows relationships **between** functions
- Simple technique, which means that the whole team can be involved in the process
- Provides a common understanding of the system
- Customers can also get involved
- Oversights and errors can be detected early
- Identifies the external parts of the system

DISADVANTAGES

- Difficult to verify completeness
- Diagrams can get complicated if not broken down into separate levels properly
- **Functions and data are treated as separate**



Group Exercise

- Work in groups
- Construct a high level FFD for an email system

(10 minutes)

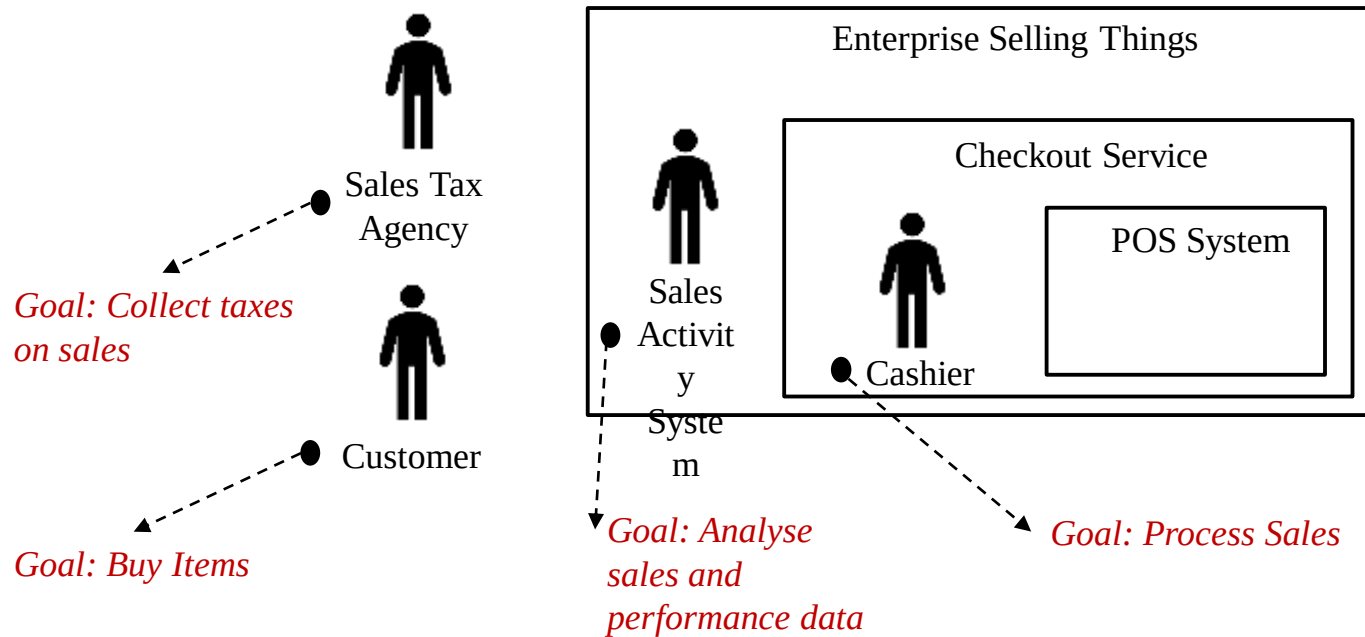


Use Cases

Use Cases

- Collection of success and failures scenarios
- Scenarios describe an actor to support a goal
- The Use Case Diagram takes a user-oriented approach to systems analysis to help capture requirements.
- **Users (actors)** are identified along with the **tasks (use cases)** that they must undertake within the system

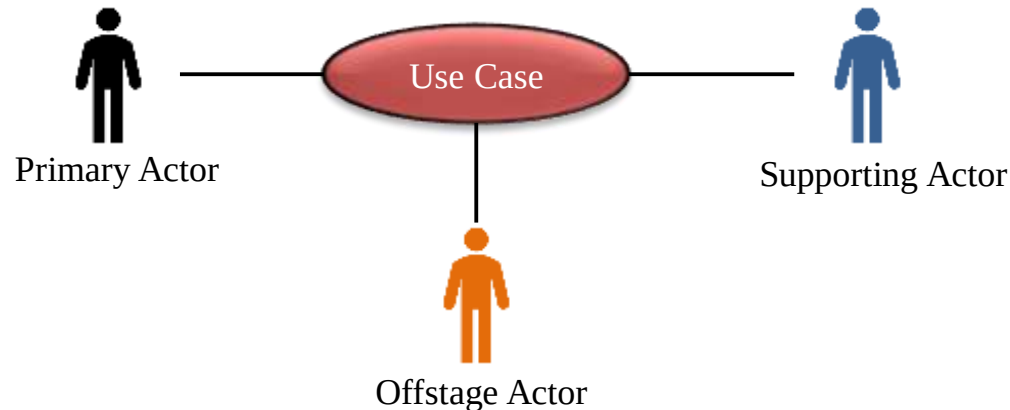
Point of Sale (POS) System



What are three kinds of Actors?

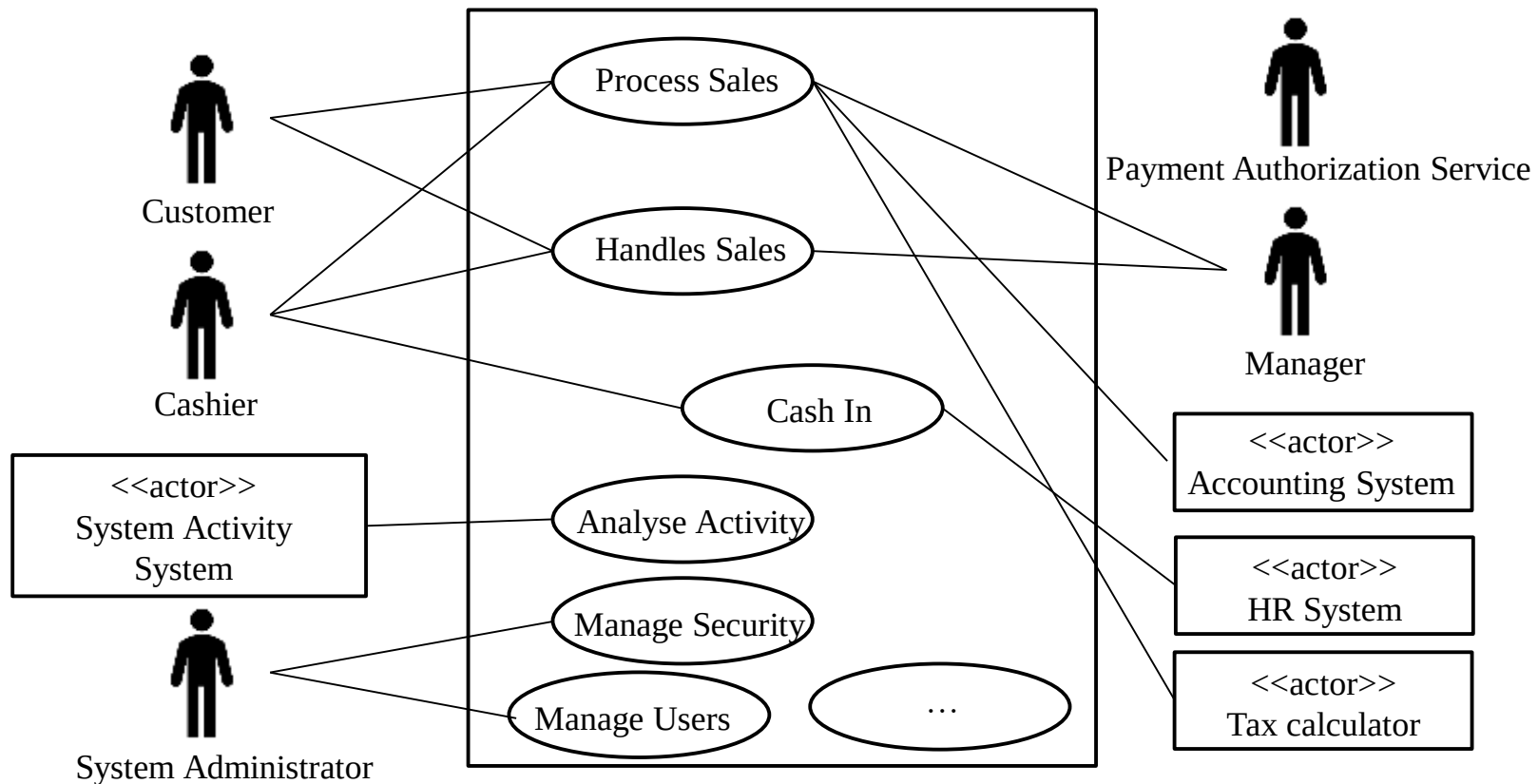
- stakeholder that initiates an interaction with the system to achieve a goal.
- Why Identify? To find user goals, which drive the use cases.

- a user who provides a service (e.g., information) to the system.
- Why Identify? To clarify external interfaces and protocols.



- a user who has an interest in the behaviour of the use case but is not the primary or supporting actor.
- Why Identify? To ensure that all the necessary interests are identified and satisfied.

Partial use case context diagram



What are main common use case formats?

- **Brief Format of a Use Case**

- One paragraph summary, usually of the main success scenario. For example;

- **Process Sale:** *A customer arrives at a checkpoint with items to purchase. The cashier uses the point-of-sale (POS) system to record each purchased item. The system presents a running total and line-item details. The customer enters payment information, which the system validates and records. The system updates inventory. The customer receives a receipt from the system and then leaves with the items.*

- **Casual Format of Use Case**

- Multiple paragraphs that covers many scenarios, e.g.

- **Handles Returns:**

- Main Success Scenarios: *A customer arrives at....*
- Alternate Scenarios: *If the customer paid by credit.....*

Fully Dressed Use case

Use Case Section	Comment
Use Case Name	Start with a verb.
Scope	The system under design.
Level	"user-goal" or "subfunction"
Primary Actor	Calls on the system to deliver its services.
Stakeholders and Interests	Who cares about this use case, and what do they want?
Preconditions	What must be true on start, <i>and</i> worth telling the reader?
Success Guarantee	What must be true on successful completion, <i>and</i> worth telling the reader.
Main Success Scenario	A typical, unconditional happy path scenario of success.
Extensions	Alternate scenarios of success or failure.
Special Requirements	Related non-functional requirements.
Technology and Data Variations List	Varying I/O methods and data formats.
Frequency of Occurrence	Influences investigation, testing, and timing of implementation.
Miscellaneous	Such as open issues.

Fully Dressed Use case – Process Sale

Use Case UC1: Process Sale

Scope: NextGen POS application

Level: user goal

Primary Actor: Cashier

Stakeholders and Interests:

- Cashier: Wants accurate, fast entry, and no payment errors, as cash drawer shortages are deducted from his/her salary.
- Salesperson: Wants sales commissions updated.
- Customer: Wants purchase and fast service with minimal effort. Wants easily visible display of entered items and prices. Wants proof of purchase to support returns.
- Company: Wants to accurately record transactions and satisfy customer interests. Wants to ensure that Payment Authorization Service payment receivables are recorded. Wants some fault tolerance to allow sales capture even if server components (e.g., remote credit validation) are unavailable. Wants automatic and fast update of accounting and inventory.
- Manager: Wants to be able to quickly perform override operations, and easily debug Cashier problems.
- Government Tax Agencies: Want to collect tax from every sale. May be multiple agencies, such as national, state, and county.
- Payment Authorization Service: Wants to receive digital authorization requests in the correct format and protocol. Wants to accurately account for their payables to the store.

Preconditions: Cashier is identified and authenticated.

Success Guarantee (or Postconditions): Sale is saved. Tax is correctly calculated. Accounting and Inventory are updated. Commissions recorded. Receipt is generated. Payment authorization approvals are recorded.

Main Success Scenario (or Basic Flow):

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total. Price calculated from a set of price rules.

Cashier repeats steps 3-4 until indicates done.

5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.
8. System logs completed sale and sends sale and payment information to the external Accounting system (for accounting and commissions) and Inventory system (to update inventory).
9. System presents receipt.
10. Customer leaves with receipt and goods (if any).

Extensions (or Alternative Flows):

*a. At any time, Manager requests an override operation:

- 1.** System enters Manager-authorized mode.

- Manager or Cashier performs one Manager-mode operation. e.g., cash balance change, resume a suspended sale on another register, void a sale, etc.
- System reverts to Cashier-authorized mode.

*b. At any time, System fails:

To support recovery and correct accounting, ensure all transaction sensitive state and events can be recovered from any step of the scenario.

- 1.** Cashier restarts System, logs in, and requests recovery of prior state.

- System reconstructs prior state.

- 2a.** System detects anomalies preventing recovery:

- 1.** System signals error to the Cashier, records the error, and enters a clean state.

1. Cashier performs resume operation, and enters the ID to retrieve the sale.
 2. System displays the state of the resumed sale, with subtotal.
 - 2a. Sale not found.
 1. System signals error to the Cashier.
 - Cashier probably starts new sale and re-enters all items.
 - Cashier continues with sale (probably entering more items or handling payment).
- 2-4a. Customer tells Cashier they have a tax-exempt status (e.g., seniors, native peoples)
1. Cashier verifies, and then enters tax-exempt status code.
 2. System records status (which it will use during tax calculations)
- 3a. Invalid item ID (not found in system):
1. System signals error and rejects entry.
 2. Cashier responds to the error:
 - 2a. There is a human-readable item ID (e.g., a numeric UPC):
 1. Cashier manually enters the item ID.
 - System displays description and price.
 - 2a. Invalid item ID: System signals error. Cashier tries alternate method.
 - 2b. There is no item ID, but there is a price on the tag:
 1. Cashier asks Manager to perform an override operation.
 - Managers performs override.
 - Cashier indicates manual price entry, enters price, and requests standard taxa for this amount (because there is no product information, the tax engine can't otherwise deduce how to tax it)

2c. Cashier performs Find Product Help to obtain true item ID and price.

2d. Otherwise, Cashier asks an employee for the true item ID or price, and does either manual ID or manual price entry (see above).

3b. There are multiple of same item category and tracking unique item identity not important (e.g., 5 packages of veggie-burgers):

1. Cashier can enter item category identifier and the quantity.

3c. Item requires manual category and price entry (such as flowers or cards with a price on them):

1. Cashier enters special manual category code, plus the price.

3-6a: Customer asks Cashier to remove (i.e., void) an item from the purchase:

This is only legal if the item value is less than the void limit for Cashiers, otherwise a Manager override is needed.

1. Cashier enters item identifier for removal from sale.
2. System removes item and displays updated running total.

2a. Item price exceeds void limit for Cashiers:

1. System signals error, and suggests Manager override.
- Cashier requests Manager override, gets it, and repeats operation.

3-6b. Customer tells Cashier to cancel sale:

1. Cashier cancels sale on System.

3-6c. Cashier suspends the sale:

1. System records sale so that it is available for retrieval on any POS register.
2. System presents a "suspend receipt" that includes the line items, and a sale ID used to retrieve and resume the sale.

and is offered a lower price):

1. Cashier requests approval from Manager.
2. Manager performs override operation.
3. Cashier enters manual override price.
4. System presents new price.

5a. System detects failure to communicate with external tax calculation system service:

1. System restarts the service on the POS node, and continues.
 - 1a. System detects that the service does not restart.

1. System signals error.

- Cashier may manually calculate and enter the tax, or cancel the sale.

5b. Customer says they are eligible for a discount (e.g., employee, preferred customer):

1. Cashier signals discount request.
2. Cashier enters Customer identification.
3. System presents discount total, based on discount rules.

5c. Customer says they have credit in their account, to apply to the sale:

1. Cashier signals credit request.
2. Cashier enters Customer identification.
3. Systems applies credit up to price=0, and reduces remaining credit.

6a. Customer says they intended to pay by cash but don't have enough cash:

1. Cashier asks for alternate payment method.
 - 1a. Customer tells Cashier to cancel sale. Cashier cancels sale on System.

7a. Paying by cash:

1. Cashier enters the cash amount tendered.
2. System presents the balance due, and releases the cash drawer.
3. Cashier deposits cash tendered and returns balance in cash to Customer.
4. System records the cash payment.

7b. Paying by credit:

1. Customer enters their credit account information.
2. System displays their payment for verification.
3. Cashier confirms.

3a. Cashier cancels payment step:

1. System reverts to "item entry" mode.

- System sends payment authorization request to an external Payment Authorization Service System, and requests payment approval.

4a. System detects failure to collaborate with external system:

1. System signals error to Cashier.

- Cashier asks Customer for alternate payment.

- System receives payment approval, signals approval to Cashier, and releases cash drawer (to insert signed credit payment receipt).

5a. System receives payment denial:

1. System signals denial to Cashier.

- Cashier asks Customer for alternate payment.

5b. Timeout waiting for response.

1. System signals timeout to Cashier.
2. Cashier may try again, or ask Customer for alternate payment.

- System records the credit payment, which includes the payment approval.
- System presents credit payment signature input mechanism.
- Cashier asks Customer for a credit payment signature. Customer enters signature.
- If signature on paper receipt, Cashier places receipt in cash drawer and closes it.

7c. Paying by check...

7d. Paying by debit...

7e. Cashier cancels payment step:

1. System reverts to "Item entry" mode.

7f. Customer presents coupons:

1. Before handling payment, Cashier records each coupon and System reduces price if appropriate. System records the used coupons for accounting reasons.

1a. Coupon entered is not for any purchased item:

1. System signals error to Cashier.

9a. There are product rebates:

1. System presents the rebate forms and rebate receipts for each item with a rebate.

9b. Customer requests gift receipt (no prices visible):

1. Cashier requests gift receipt and System presents it.

9c. Printer out of paper.

1. If System can detect the fault, will signal the problem.
2. Cashier replaces paper.
3. Cashier requests another receipt.

Special Requirements:

- Credit authorization response within 30 seconds 90% of the time.
- Somehow, we want robust recovery when access to remote services such the inventory system is failing.
- Language internationalization on the text displayed.
- Pluggable business rules to be insertable at steps 3 and 7.
- ...

Technology and Data Variations List:

*a. Manager override entered by swiping an override card through a card reader, or entering an authorization code via the keyboard.

3a. Item identifier entered by bar code laser scanner (if bar code is present) or keyboard.

3b. Item identifier may be any UPC, EAN, JAN, or SKU coding scheme.

7a. Credit account information entered by card reader or keyboard.

7b. Credit payment signature captured on paper receipt. But within two years, we predict many customers will want digital signature capture.

Frequency of Occurrence: Could be nearly continuous.

Open Issues:

- What are the tax law variations?
- Explore the remote service recovery issue.
- What customization is needed for different businesses?
- Must a cashier take their cash drawer when they log out?
- Can the customer directly use the card reader, or does the cashier have to do it?

Other Requirements

- Supplementary Specification
 - Captures and identified other kinds of requirements such as reports, documentation, packaging, supportability, licensing and so forth.
- Glossary
 - Captures terms and definitions; it can play role of data dictionary
- Vision
 - Serves to tersely communicate the big idea.
- Business Rules
 - Captures long living and spanning rules or policies, such as tax laws.

Summary

- Different tools and techniques can be used to elicit software requirements
- Systemic Textual Analysis, Viewpoint Analysis, Functional Modelling and Use Cases are some of these tools and techniques

Thanks

Next Lecture:

Object Oriented Analysis and Design